POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

SYLABUS PRZEDMIOTU

Nazwa przedmiotu:	**Zarządzanie infrastrukturą chmurową**
Kod przedmiotu:	ZIC
Kierunek / Profil:	Informatyka / praktyczny
Tryb studiów:	stacjonarny
Rok / Semestr:	4 / 7
Charakter:	obieralny
Odpowiedzialny:	dr inż. Adam Urbanowicz
Wersja z dnia:	26.02.2026

1. Godziny zajęć i punkty ECTS

Wykłady	Ćwiczenia	Laboratori	Z prowa-dzącym	Praca własna	Łącznie	ECTS
30 h	—	30 h	60 h	65 h	125 h	**5**

Zadania do samodzielnej realizacji: Definiowanie infrastruktury jako kodu z użyciem Terraform i Ansible; praca z LocalStack i OpenStack; przygotowanie sprawozdań i projektu końcowego dotyczącego wdrożenia infrastruktury chmurowej.

2. Forma zajęć

Forma zajęć	Sposób zaliczenia
Wykład	Egzamin
Laboratorium	Zaliczenie z oceną

3. Cel dydaktyczny

Celem przedmiotu jest zapoznanie studentów z metodologią Infrastructure as Code (IaC) oraz narzędziami i platformami umożliwiającymi automatyczne tworzenie, konfigurowanie

i zarządzanie infrastrukturą chmurową na skalę produkcyjną. Student nabywa praktyczne umiejętności pracy z Terraform (provisionowanie zasobów), Ansible (zarządzanie konfiguracją i automatyzacja), Amazon Web Services (usługi chmury publicznej) oraz OpenStack i jego lokalnym emulatorem LocalStack (chmura prywatna i środowiska testowe). Przedmiot kształci kompetencje inżyniera DevOps/Cloud, który potrafi projektować powtarzalne, wersjonowane i audytowalne środowiska infrastrukturalne.

Wersja w języku angielskim (English version)

The aim of the course is to familiarize students with the Infrastructure as Code (IaC) methodology and the tools and platforms enabling the automatic creation, configuration, and management of cloud infrastructure at production scale. Students acquire practical skills in working with Terraform (resource provisioning), Ansible (configuration management and automation), Amazon Web Services (public cloud services), and OpenStack together with its local emulator LocalStack (private cloud and testing environments). The course develops the competencies of a DevOps/Cloud engineer who can design repeatable, versioned, and auditable infrastructure environments.

4. Przedmioty wprowadzające

Przedmiot	Wymagane zagadnienia
KSI – Konteneryzacja serwisów internetowych	Znajomość Dockera, Docker Compose i podstaw Kubernetes; umiejętność pracy z plikami YAML i obsługi terminala Linux.
AMM – Architektura mikroserwisowa i mikrofrontendowa	Rozumienie architektury systemów rozproszonych, pojęć sieci, load balancingu i wdrożeń wieloserwisowych.
SOP – Systemy operacyjne	Sprawna obsługa systemu Linux: zarządzanie procesami, uprawnienia, SSH, skrypty powłoki.

5. Treści programowe

Nr zaj.	Wykład	Ćwiczenia / Laboratorium / Pracownia

Nr zaj.	Wykład	Ćwiczenia / Laboratorium / Pracownia
1.	Infrastructure as Code – motywacja, historia i korzyści. Podejście deklaratywne vs. imperatywne. Przegląd narzędzi IaC: Terraform, Ansible, Pulumi, CloudFormation. Miejsce IaC w cyklu DevOps.	
2.	Terraform – architektura i podstawy. Providers (AWS, OpenStack), resources, data sources. Język HCL: typy danych, wyrażenia, funkcje wbudowane. Cykl życia konfiguracji: init, validate, plan, apply, destroy.	
3.	Terraform state: cel i struktura pliku stanu. Zdalny backend (S3 + DynamoDB lock). Import istniejących zasobów. Terraform workspace – izolacja środowisk (dev/staging/prod).	
4.	Moduły Terraform: tworzenie, parametryzacja i kompozycja modułów. Rejestr publiczny (Terraform Registry). Wzorce organizacji kodu: flat, monorepo, multi-repo. Refaktoryzacja konfiguracji.	
5.	AWS – globalna infrastruktura i model usług. VPC: subnety publiczne i prywatne, Internet Gateway, NAT Gateway, routing tables, security groups, Network ACL. Provisionowanie sieci w Terraform.	
6.	AWS EC2 i zarządzanie obliczeniami: typy instancji, AMI, key pairs, user data, Elastic IP. Auto Scaling Groups i Launch Templates. Elastic Load Balancer (ALB/NLB). Konfiguracja przez Terraform.	
7.	AWS – usługi zarządzane: RDS (PostgreSQL/MySQL), ElastiCache (Redis), S3 (bucket policies, versioning, lifecycle), SQS, SNS. Wdrożenie warstwy danych aplikacji przez Terraform.	

Nr zaj.	Wykład	Ćwiczenia / Laboratorium / Pracownia
8.	Ansible – architektura i instalacja. Inventory (statyczne i dynamiczne). Ad-hoc commands. Playbooks: plays, tasks, handlers, notify. Moduły wbudowane (apt, yum, copy, template, service, shell).	
9.	Ansible – zaawansowane techniki: role (struktura katalogów, defaults, vars, tasks, handlers, templates), Ansible Galaxy, zmienne grupowe i hostowe, Vault (szyfrowanie sekretów). Idempotentność.	
10.	Integracja Terraform + Ansible: przekazywanie outputów Terraform do inventory Ansible (dynamic inventory), null_resource i local-exec/remote-exec provisioners. Wzorce automatyzacji end-to-end.	
11.	LocalStack – emulacja usług AWS lokalnie. Instalacja i konfiguracja (Docker Compose). Obsługiwane usługi: S3, SQS, SNS, Lambda, DynamoDB, EC2, IAM. Konfiguracja Terraform provider dla LocalStack. Strategie testowania infrastruktury.	
12.	OpenStack – architektura i komponenty: Keystone (auth), Nova (compute), Neutron (network), Cinder (block storage), Glance (images), Swift (object storage), Heat (orchestration). Porównanie z usługami AWS.	
13.	Praca z OpenStack: openstack CLI, zarządzanie instancjami, sieciami, woluminami i floating IP. Szablony Heat (HOT): struktura, parametry, resources, outputs. Automatyzacja przez Terraform provider OpenStack.	
14.	Bezpieczeństwo infrastruktury chmurowej: IAM – użytkownicy, grupy, role, polityki (managed, inline), zasada least privilege. AWS Secrets Manager i HashiCorp Vault. Szyfrowanie EBS/S3 (KMS). Audyt z AWS CloudTrail.	

Nr zaj.	Wykład	Ćwiczenia / Laboratorium / Pracownia
15.	Monitoring i obserwowalność: CloudWatch (metryki, logi, alarmy, dashboardy), AWS Cost Explorer i budżety. Eksport metryk do Prometheus, wizualizacja w Grafana. CI/CD dla infrastruktury: terraform plan w GitHub Actions, atlantis, automatyczne wdrożenia po merge. Podsumowanie kursu i ścieżki certyfikacji (AWS Cloud Practitioner, AWS Solutions Architect Associate, Terraform Associate).	

6. Przedmiotowe efekty uczenia się

Wiedza

- Student zna i rozumie koncepcję Infrastructure as Code (IaC): zasady deklaratywnego i imperatywnego podejścia do definiowania infrastruktury, korzyści z wersjonowania i powtarzalności środowisk.
- Student zna i rozumie architekturę i model działania Terraform: providers, resources, data sources, state (lokalny i zdalny), moduły, workspace oraz cykl plan–apply–destroy.
- Student zna i rozumie zasady zarządzania konfiguracją z użyciem Ansible: architektura (control node, managed nodes, inventory), playbooks, role, zmienne, szablony Jinja2 i idempotentność operacji.
- Student zna i rozumie model usług chmurowych (IaaS, PaaS, SaaS) oraz globalną infrastrukturę AWS: regiony, strefy dostępności, VPC, subnety, grupy bezpieczeństwa.
- Student zna i rozumie kluczowe usługi AWS wykorzystywane w automatyzacji infrastruktury: EC2, S3, RDS, IAM, ELB, Auto Scaling, Route 53, CloudWatch, ECR/ECS.
- Student zna i rozumie architekturę OpenStack: komponenty (Nova, Neutron, Cinder, Glance, Keystone, Swift, Heat) i ich odpowiedniki w usługach AWS.
- Student zna i rozumie przeznaczenie i możliwości LocalStack jako lokalnego emulatora usług AWS, umożliwiającego testowanie infrastruktury bez dostępu do chmury publicznej i bez kosztów.
- Student zna i rozumie zasady bezpiecznego zarządzania dostępem w chmurze: IAM (użytkownicy, role, polityki, least privilege), zarządzanie kluczami i sekretami (AWS Secrets Manager, HashiCorp Vault), szyfrowanie zasobów.
- Student zna i rozumie strategie zarządzania stanem infrastruktury: remote state (S3 + DynamoDB), state locking, import istniejących zasobów, techniki refaktoryzacji konfiguracji Terraform.
- Student zna i rozumie podstawy monitorowania i obserwowalności infrastruktury chmurowej: metryki, logi, alerty (CloudWatch, Prometheus, Grafana) oraz koncepcję

SLI/SLO/SLA.

Umiejętności

- Student potrafi zainstalować i skonfigurować środowisko pracy z Terraform, Ansible i LocalStack oraz nawiązać połączenie z klastrem OpenStack.
- Student potrafi napisać konfigurację Terraform definiującą sieć (VPC, subnety, routing, security groups), maszyny wirtualne oraz zarządzane usługi bazodanowe w AWS lub ich odpowiedniki w LocalStack.
- Student potrafi stosować moduły Terraform do organizowania i wielokrotnego użycia konfiguracji infrastruktury, zarządzać stanem zdalnym w S3 z blokadą w DynamoDB.
- Student potrafi napisać playbook Ansible instalujący i konfigurujący serwer aplikacyjny (np. Nginx + Node.js/Python) na zdalnych hostach, stosując role, zmienne i szablony Jinja2.
- Student potrafi zintegrować Terraform z Ansible w celu automatycznego provisionowania maszyn i ich konfigurowania w jednym potoku wdrożeniowym.
- Student potrafi uruchomić LocalStack i za pomocą AWS CLI oraz Terraform przetestować lokalnie infrastrukturę obejmującą S3, SQS, Lambda, DynamoDB i EC2.
- Student potrafi tworzyć i zarządzać zasobami OpenStack (instancje, sieci, woluminy, floating IP, security groups) z poziomu CLI (openstack client) oraz za pomocą szablonów Heat.
- Student potrafi skonfigurować zarządzanie dostępem IAM w AWS: tworzyć użytkowników, role i polityki uprawnień zgodnie z zasadą najmniejszych uprawnień.
- Student potrafi skonfigurować monitoring infrastruktury: tworzyć alarmy CloudWatch, eksportować metryki do Prometheus i wizualizować je w Grafana.
- Student potrafi zbudować potok CI/CD (GitHub Actions) automatycznie weryfikujący (terraform plan, ansible-lint) i wdrażający zmiany infrastruktury po zatwierdzeniu pull requesta.

Kompetencje społeczne

- Student jest gotów do traktowania infrastruktury jako kodu – przechowywania konfiguracji w systemie kontroli wersji, przeprowadzania code review zmian infrastrukturalnych i stosowania standardów branżowych (formatowanie, dokumentacja modułów).
- Student jest gotów do świadomego zarządzania kosztami chmury publicznej: rozumie model rozliczeń pay-as-you-go, potrafi szacować koszty planowanych zasobów i stosować mechanizmy ograniczania wydatków (budgets, reserved instances, auto scaling).
- Student jest gotów do samodzielnego pogłębiania wiedzy z zakresu dynamicznie rozwijającego się ekosystemu chmurowego, śledzenia dokumentacji dostawców (AWS, OpenStack, HashiCorp) i certyfikowania swoich kompetencji (AWS Cloud Practitioner, Terraform Associate).

7. Kryteria oceny

Wykład:

- egzamin pisemny z zadaniami problemowymi

Ćwiczenia / Laboratorium:

- raport z wykonanego zadania
- prezentacja projektu i dokumentacji
- obrona projektu

8. Metody dydaktyczne

Wykład:

- wykład z prezentacją multimedialną
- wykład z prezentacją oprogramowania

Ćwiczenia / Laboratorium:

- metoda projektów (projekt praktyczny)
- rozwiązywanie zadań
- warsztaty

9. Literatura

Podstawowa:

- Y. Brikman: "Terraform. Krótkie wprowadzenie. Wdrażanie infrastruktury jako kodu", Helion, Gliwice 2023 (Terraform: Up & Running, 3rd ed., O'Reilly 2022).
- dokumentacja oficjalna Terraform: developer.hashicorp.com/terraform/docs.
- dokumentacja oficjalna Ansible: docs.ansible.com.
- dokumentacja oficjalna AWS: docs.aws.amazon.com.
- dokumentacja LocalStack: docs.localstack.cloud.
- dokumentacja OpenStack: docs.openstack.org.

Uzupełniająca:

- K. Morris: "Infrastructure as Code. Dynamic Systems for the Cloud Age", 2nd ed., O'Reilly 2021.
- J. Looney, M. Domański: "Ansible w praktyce. Automatyzacja konfiguracji i wdrożeń", Helion, Gliwice 2022 (Ansible: Up and Running, 3rd ed., O'Reilly 2022).
- B. Burns, J. Beda, K. Hightower: "Kubernetes. Tworzenie niezawodnych systemów rozproszonych", Helion, Gliwice 2023.
- A. Wittig, M. Wittig: "Amazon Web Services w akcji", Helion, Gliwice 2023 (Amazon Web Services in Action, 3rd ed., Manning 2023).